Curso SQL

Comisión 81810
Alumno: Nicolás Pereyra

Proyecto: Scooter rental business database

Introducción

El presente proyecto desarrolla el diseño y la estructura de una base de datos relacional para una startup de movilidad sostenible que se especializa en el alquiler de monopatines eléctricos en zonas metropolitanas. El sistema busca centralizar el control operativo de la flota y el registro transaccional de los usuarios. Esta infraestructura de datos es fundamental para transformar estos registros brutos en indicadores de rendimiento (KPIs), permitiendo a la organización evaluar su estado actual y realizar proyecciones de rentabilidad basadas en datos históricos de consumo y operatividad.

Situación Problemática

Actualmente, la empresa carece de un sistema integrado que permita rastrear la trazabilidad de los viajes y la usabilidad de los activos (scooters). Esta desorganización genera brechas en el análisis de la clientela y en el control financiero. La implementación de esta base de datos resuelve la necesidad de estructurar métricas críticas como la frecuencia de uso por estación, el flujo de ingresos por métodos de pago y la gestión de egresos operativos, permitiendo así una visión clara de la rentabilidad del modelo de negocio.

Objetivo:

Centralizar la información operativa y transaccional para optimizar la toma de decisiones en tres áreas clave:

- Logística: Medir la usabilidad de las estaciones y el estado de la flota (batería y mantenimiento).
- Analítica: Identificar patrones de uso (frecuencia, horarios pico y rutas comunes).
- Contable: Controlar la facturación, los montos recaudados por periodo y la efectividad de los métodos de pago.

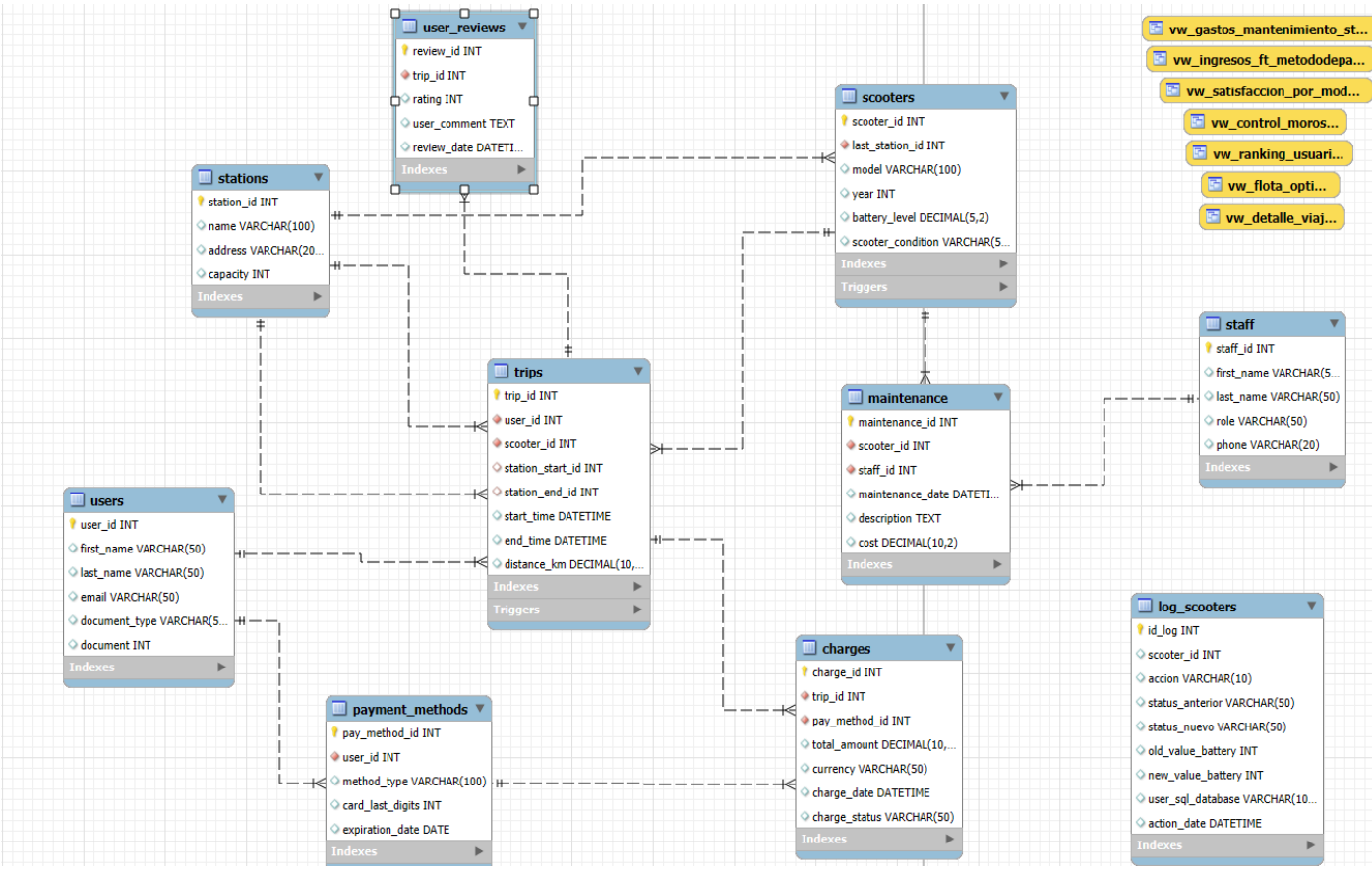
Tablas y tipos de datos

Tabla	Descripción	Campos	Tipo de Dato
Stations	Ubicaciones	station_id (PK) / name / address / capacity	INT / VARCHAR / VARCHAR / INT

	físicas		
Users	Clientes registrados	user_id (PK) / first_name / last_name / email (unique) / document_type / document	INT / VARCHAR / VARCHAR / VARCHAR / VARCHAR / INT
Scooters	Flota de vehículos	scooter_id (PK) / last_station_id (FK) / model / year / battery_level / scooter_condition	INT / INT / VARCHAR / INT / DECIMAL / INT / VARCHAR
Trips	Viajes realizados	trip_id (PK) / user_id (FK) / scooter_id (FK) / station_start_id (FK) / station_end_id (FK) / start_time / end_time / distance_km	INT / INT / INT / INT / INT / DATETIME / DATETIME / DECIMAL
Payment Methods	Método de pagos	pay_method_id (PK) / user_id (FK) / method_type / card_last_digits / expiration_date	INT / INT / VARCHAR / INT / DATE
Charges	Facturación	charge_id (PK) / trip_id (FK) / pay_method_id (FK) / total_amount / currency / charge_date / charge_status	INT / INT / INT / DECIMAL / VARCHAR / DATETIME / VARCHAR
Maintenance	Registro de mantenimiento de flota	maintenance_id (PK) / scooter_id (FK) / staff_id (FK) / maintenance_date / description / cost	INT / INT / INT / DATETIME / TEXT / DECIMAL
Staff	Personal de la empresa	staff_id (PK) / first_name / last_name / role / phone	INT / VARCHAR / VARCHAR / VARCHAR / VARCHAR
User_reviews	Satisfacción del cliente	review_id (PK) / trip_id / rating / user_comment / review_date	INT / INT / INT / TEXT / DATETIME
Log_scooters	Registro de datos de auditoría de la flota	id_log (PK) / scooter_id (FK) / accion / status_anterior / status_nuevo / old_value_battery / new_value_battery / user_sql_database / action_date	INT / INT / VARCHAR / VARCHAR / VARCHAR / INT / INT /VARCHAR / DATETIME

<u>Relaciones entre tablas:</u>

- Station 1 a N Scooters (Para cada estación habrá muchos scooters estacionados para utilizar)
- Station 1 a Trips (Una estación puede formar parte de muchos viajes)
- Scooters 1 a N a Trips (Un monopatín puede ser utilizado en muchos viajes)
- Users 1 a N a Trips (Un usuario puede hacer muchos viajes)
- Users 1 a N a Payment Methods (Un usuario puede utilizar varios métodos de pago)
- Charges 1 a 1 Trips (Habrá un solo cargo por viaje)
- Payment Methods 1 a N Charges (Un método de pago se usa para muchos cargos)
- Scooters 1 a N Maintenance (Un scooter puede recibir varios mantenimientos)
- Staff 1 a N Maintenance (Un personal del staff puede realizar varios mantenimientos)
- Trips 1 a 1 User_reviews (Seria una reseña por viaje realizado)



Importación de Datos (ETL) - Scooter Rental Business Database

Introducción

Para la carga de datos en las tablas, se eligió la importación en formato CSV. Se priorizó el orden jerárquico de las tablas (relación Padre - Hijo) para asegurar la correcta integridad referencial durante el proceso.

Orden de Importación

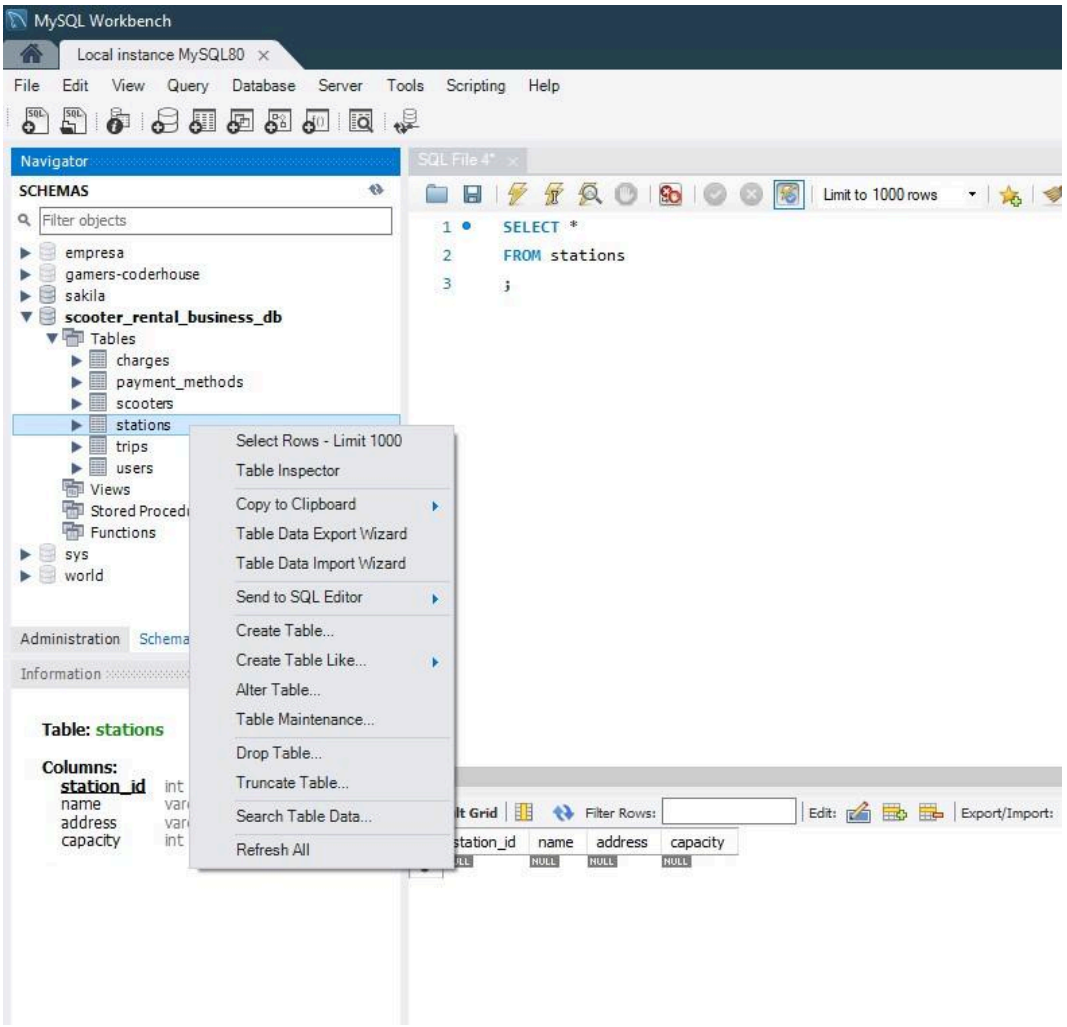
El orden de importación fue estricto debido a las restricciones impuestas por las *Foreign Keys* (*constraints*). El orden de inserción de datos fue:

1. `Stations`
2. `Users`
3. `Scooters`
4. `Payment_Methods`
5. `Trips`

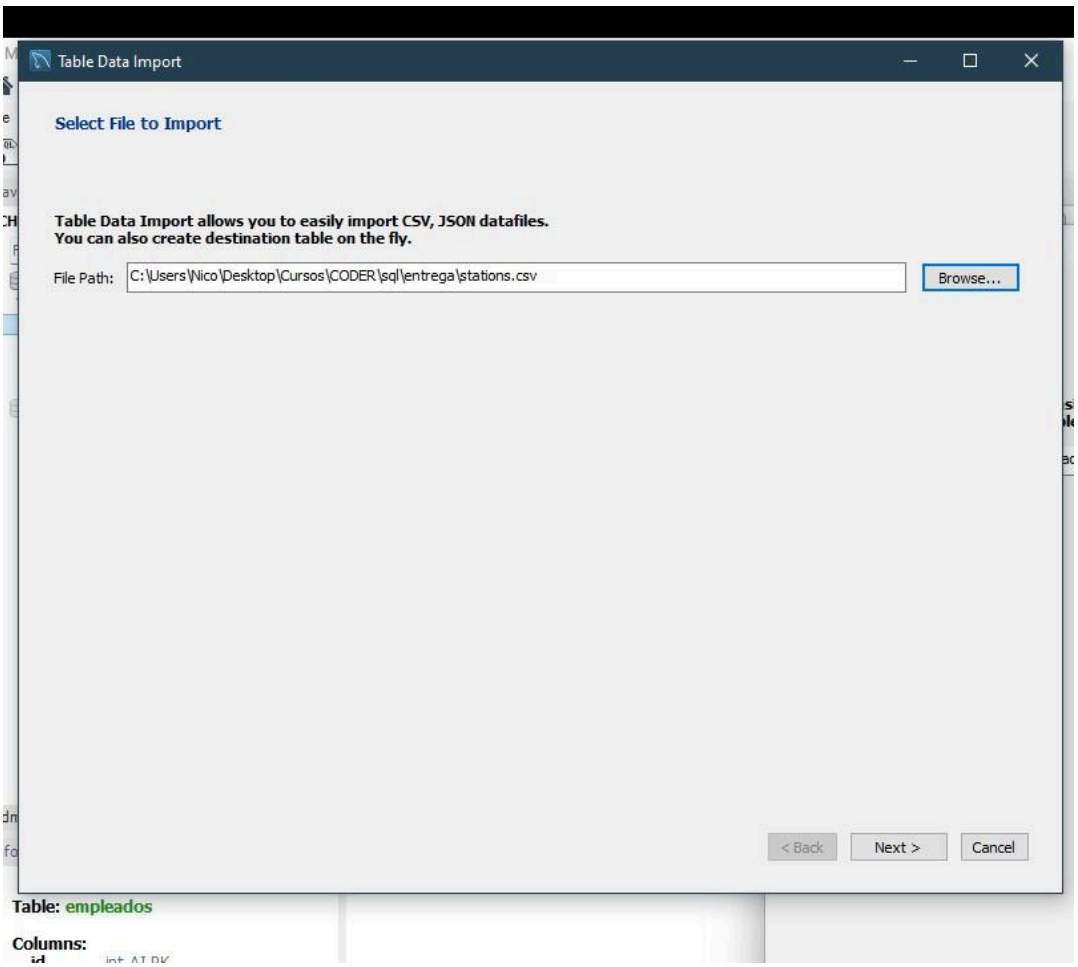
6. `Charges`
7. `Staff`
8. `Maintenance`
9. `User_reviews`

<u>Paso a paso para importar cada archivo:</u>

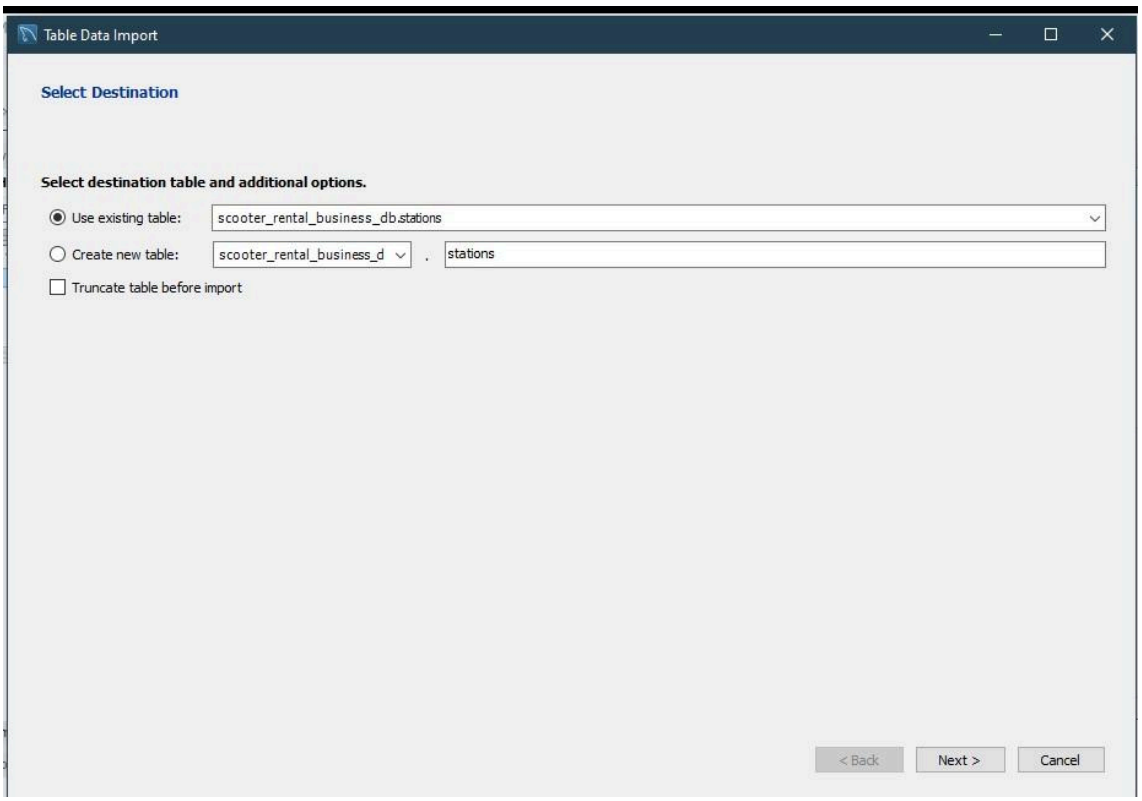
1. **Preparación del entorno:** Nos aseguramos de haber ejecutado el script de creación de tablas (`CREATE TABLE`) y que las tablas estén vacías.



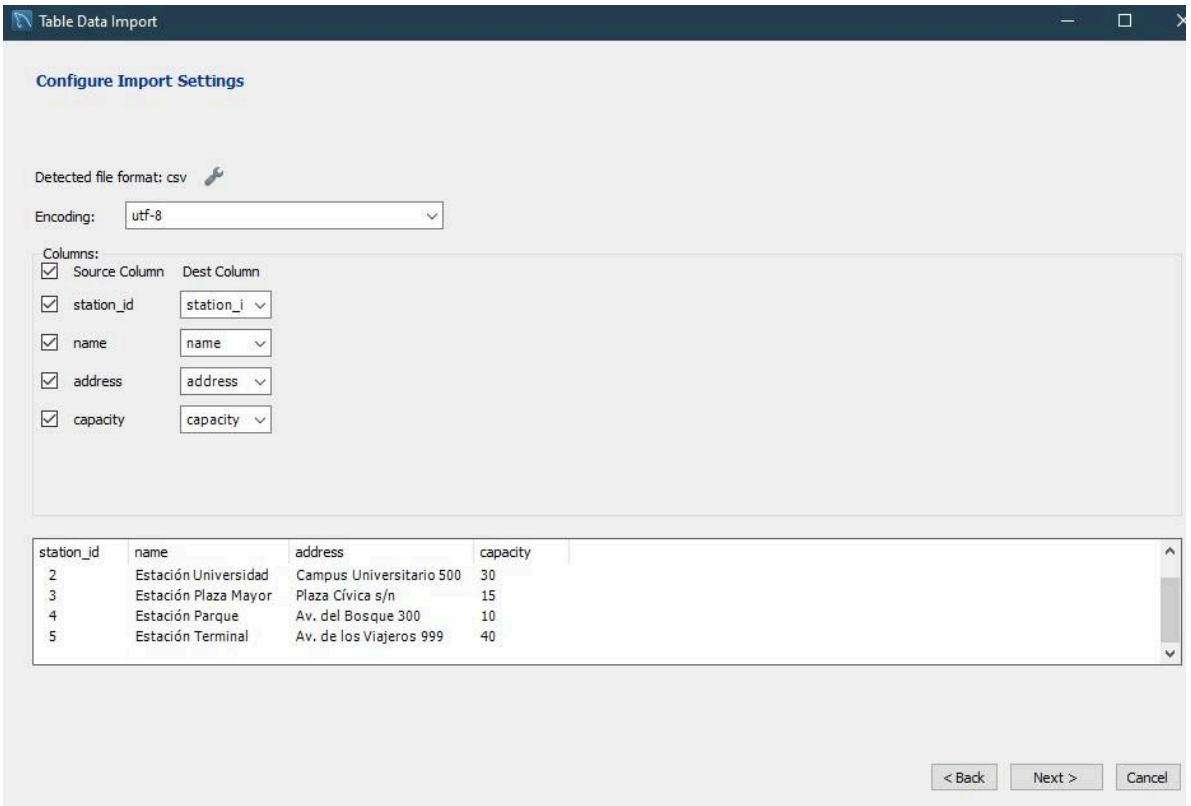
2. **Abrir el Asistente:** En el panel de navegación izquierdo (Schemas) de MySQL Workbench, se despliega la base de datos y hacer clic derecho sobre la tabla que se desea insertar los datos (ej. `Stations`).
3. **Seleccionar la herramienta de importación:** En el menú desplegable, seleccionar "Table Data Import Wizard".



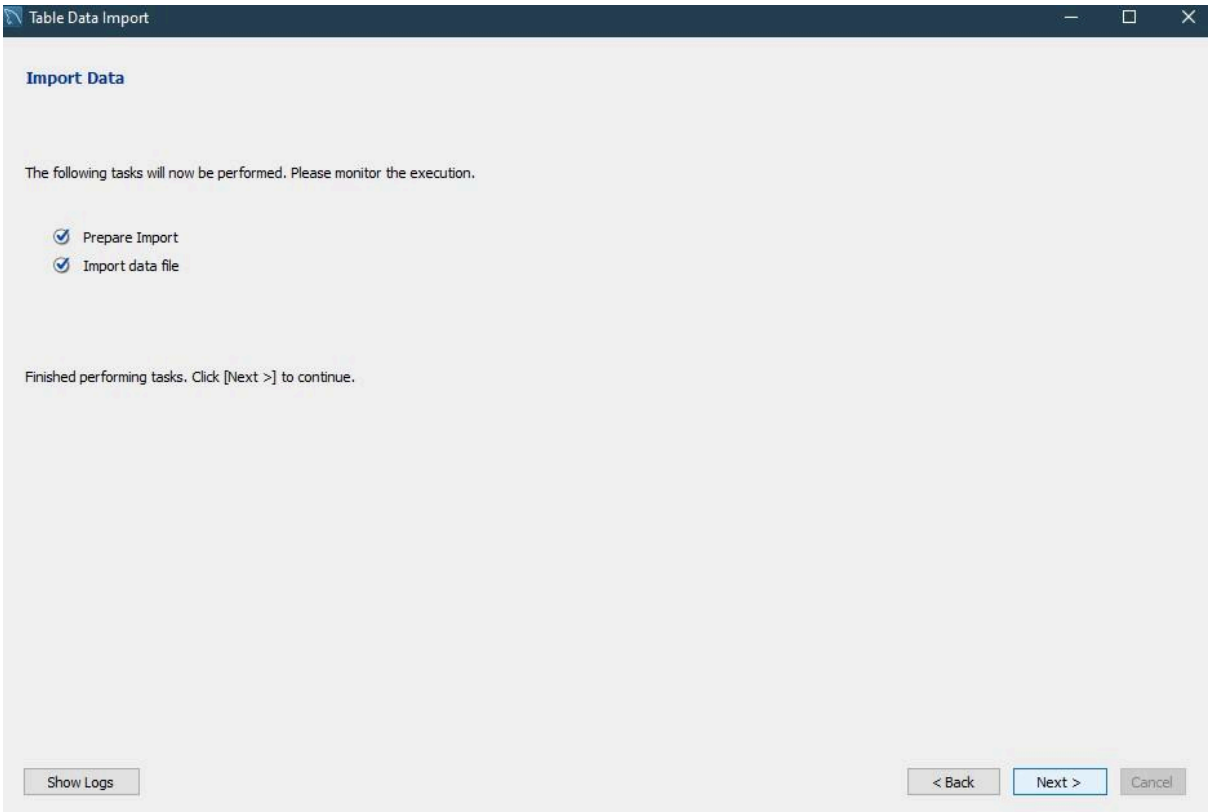
4. **Cargar el archivo:** Se abrirá una ventana. Hacer clic en "Browse" y buscar el archivo `.csv` correspondiente en la computadora (ej. `stations.csv`). Hacer clic en "Next".
5. **Configurar el destino:** Seleccionar la opción "Use existing table" (Usar tabla existente) en este caso y verificar que el nombre de la tabla seleccionada sea el correcto. Hacer clic en "Next".



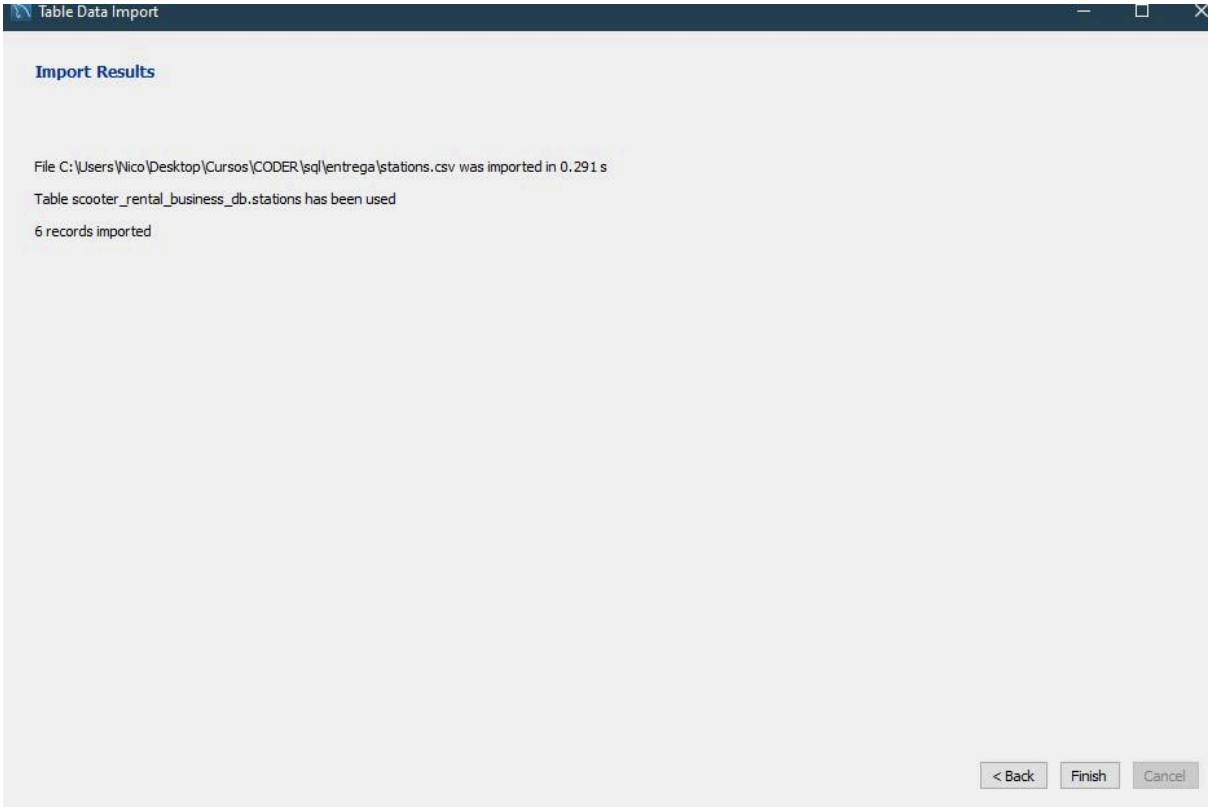
6. **Mapeo de Columnas:** En la pantalla de configuración, verificar que las columnas del archivo origen (Source Column) coincidan con las columnas de destino (Field) de la tabla en MySQL. Como los archivos CSV tienen los mismos nombres en la cabecera, el mapeo debería ser automático. Hacer clic en "Next".



7. **Ejecutar Importación:** Hacer clic en "Next" para comenzar la ejecución de las tareas. MySQL leerá el archivo e insertará los registros uno por uno.



8. **Finalizar:** Una vez que aparezca el mensaje de éxito, hacer clic en "Finish".



9. **Verificar:** Dentro de una hoja de SQL de nuestra DB ejecutar un `SELECT * FROM nombre_de_la_tabla;` para comprobar que los datos se cargaron correctamente.

station_id	name	address	capacity
1	Estación Central	Av. Siempre Viva 123	20
2	Estación Universidad	Campus Universitario 500	30
3	Estación Plaza Mayor	Plaza Cívica s/n	15
4	Estación Parque	Av. del Bosque 300	10
5	Estación Terminal	Av. de los Viajeros 999	40
6	Estación Shopping	Centro Comercial 100	25
NULL	NULL	NULL	NULL

10. **Repetir:** Se repite los pasos del 2 al 9 para el resto de los archivos respetando el orden jerárquico mencionado al principio.

Views

1) vw_flota_optima: Ver listado de solo la flota de scooters en condición óptima disponibles.
2) vw_detalle_viajes: Ver detalle descriptivo del usuario que realiza un viaje, esto nos sirve como registro detallado de la clientela.
3) vw_ingresos_ft_metododepago: Obtener el flujo de ingresos de la empresa de forma histórica estilo KPIs
4) vw_control_morosos: Esta view nos ayuda a identificar los clientes que están morosos de pago.
5) vw_ranking_usuarios: Ranking de heavy users, check de usabilidad y frecuencia.
6) vw_gastos_mantenimiento_staff: Visualizar los el mantenimiento que realizo cada empleado en materia de costo.
7) vw_satisfaccion_por_modelo: Nivel de satisfacción del cliente respecto al servicio prestado.

Functions

1) fn_duracion_viaje_minutos: En este caso creamos una función que nos traerá el total de minutos que dura cada viaje, insertando el id del viaje. Se utiliza un cálculo interno sin recurrir a otro cruce de tabla.
2) fn_estacion_actual_scooter: En esta función personalizada se consigue el paradero actual de cada scooter que indiquemos a través de su scooter_id. En este caso reemplazamos el join con una subconsulta para poder traernos la estación asignada al scooter.

Stored Procedures

1) sp_ordenar_tb_scooters: En este stored procedure se ejecuta un ordenamiento de tabla, de forma de SQL Dinámico donde asignamos la función a la variable y le damos la instrucción.
2) sp_registrar_nueva_estacion & sp_eliminar_estacion: Stored procedure de SQL estático para agregar y eliminar registros en una tabla. En el ejemplo sería una estación de scooters.

Triggers

1) trg_auditoria_cambios_scooters: Trigger Log de auditoría para registrar todos los cambios/actualizaciones sobre la tabla de scooters.
2) trg_actualizar_condicion_scooter: Trigger que cambia el estado del scooter cuando inicia un viaje de forma automática.

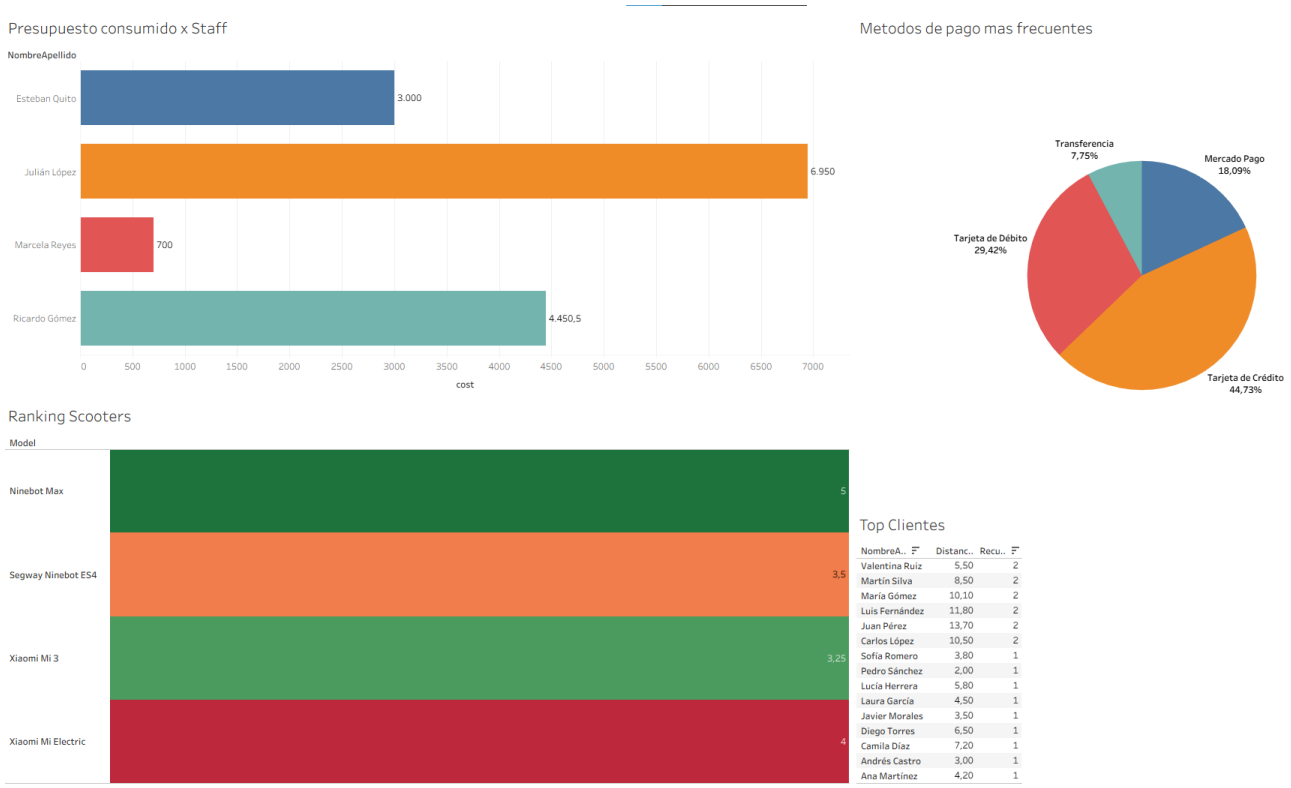
Permisos

1) Se creo 1 usuario 'usuario_auditor1'@'localhost' con permiso SELECT solo lectura sobre la base y todas sus tablas (scooter_rental_business_db.*)
2) Se creo otro usuario 'usuario_operador1'@'localhost' donde se le otorga permisos de lectura (SELECT), inserción (INSERT) y modificación (UPDATE) sobre todas las tablas. No se incluye DELETE.

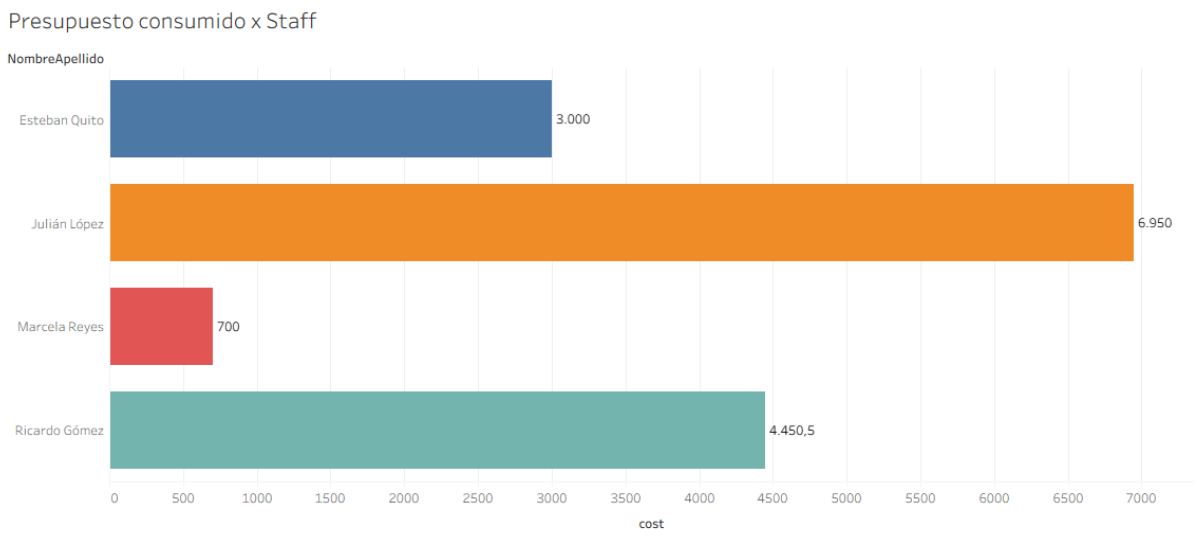
Transacciones

1) En la primera transacción se realiza la baja de scooters inactivos o dados de baja en la tabla scooters, donde una vez realizada la transacción se hace el COMMIT luego del DELETE FROM la tabla de scooters. Con posibilidad de hacer rollback previo al grabado.
2) En la segunda transacción se inserta 2 lotes de información de forma transaccional dentro de la tabla stations, el objetivo es dar de alta nuevas sucursales, en casos pesados de carga de datos se crea un primer SAVEPOINT lote_primeros_4 que guarda esa esa primera parte realizada, luego se inserta la 2da parte y se realiza un nuevo SAVEPOINT llamado lote_ultimos_4. Si quisieramos volver atras al primer guardado se pone el comando RELEASE SAVEPOINT lote_primeros_4. Sea el resultado que sea finalizamos con el COMMIT.

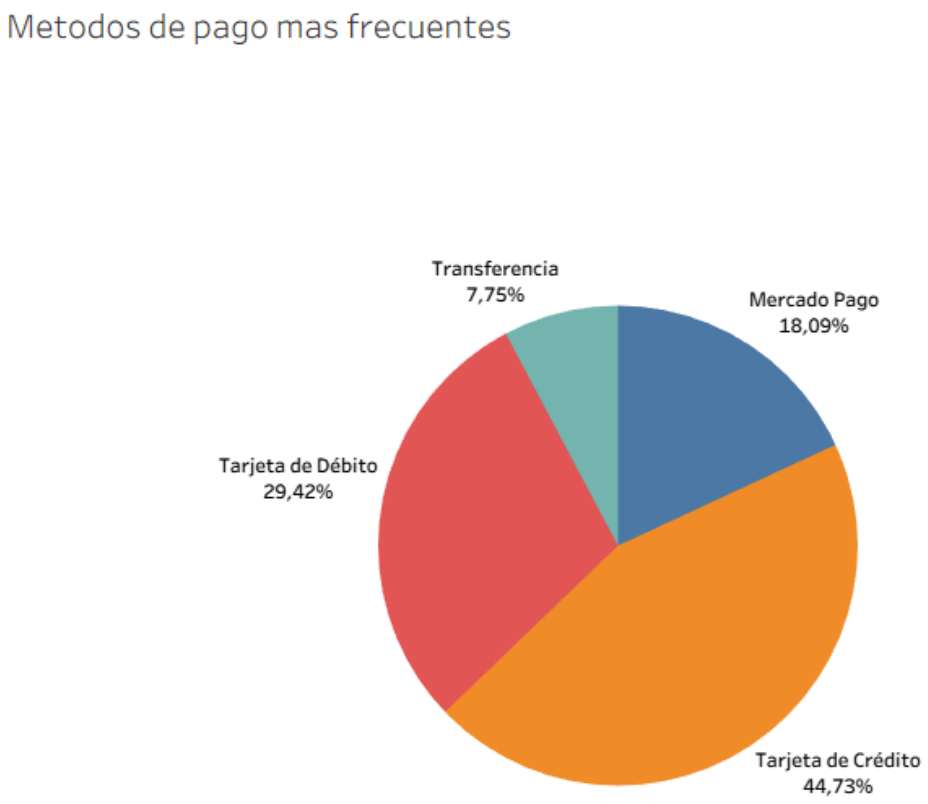
Dashboard BI



NombreA..	Distanc..	Recu..
Valentina Ruiz	5,50	2
Martín Silva	8,50	2
María Gómez	10,10	2
Luis Fernández	11,80	2
Juan Pérez	13,70	2
Carlos López	10,50	2
Sofía Romero	3,80	1
Pedro Sánchez	2,00	1
Lucía Herrera	5,80	1
Laura García	4,50	1
Javier Morales	3,50	1
Diego Torres	6,50	1
Camila Díaz	7,20	1
Andrés Castro	3,00	1
Ana Martínez	4,20	1



w_gastos_mantenimiento_staf



vw_ingresos_ft_metododepago

vw_satisfaccion_por_modelo

Top Clientes

NombreA..	Distanc..	Recu..
Valentina Ruiz	5,50	2
Martín Silva	8,50	2
María Gómez	10,10	2
Luis Fernández	11,80	2
Juan Pérez	13,70	2
Carlos López	10,50	2
Sofía Romero	3,80	1
Pedro Sánchez	2,00	1
Lucía Herrera	5,80	1
Laura García	4,50	1
Javier Morales	3,50	1
Diego Torres	6,50	1
Camila Díaz	7,20	1
Andrés Castro	3,00	1
Ana Martínez	4,20	1

vw_ranking_usuarios

Herramientas utilizadas

- MySQL
- MySQL Workbench
- Tableau
- Archivos CSV (Comma Separated Values)
- Google Docs